

# Day 5 — GROUP BY

Today is a major step.

Until Day 4, you learned how to calculate **one overall summary**:

```
SELECT COUNT(*)  
FROM documents;
```

Today you'll learn how to answer:

How many documents does each user have?

How many PDF vs Markdown documents exist?

What is the average page count for each file type?

How many documents are in each processing status?

This is where SQL starts becoming genuinely powerful for **analytics and Data Engineering**.

**Rule:** You write the SQL first. I will not give solutions before your attempt.

---

## 1. Today's Goals

By the end of Day 5, you should understand:

- What GROUP BY actually does
  - Grouping by one column
  - Grouping by multiple columns
  - GROUP BY + COUNT
  - GROUP BY + SUM
  - GROUP BY + AVG
  - GROUP BY + MIN / MAX
  - GROUP BY + WHERE
  - Why selected columns must be grouped or aggregated
  - Common GROUP BY errors
  - RAG analytics using grouping
- 

## 2. Our Database

Continue using:

**documents**

| id  | user_id | title            | file_type | page_count | status     |
|-----|---------|------------------|-----------|------------|------------|
| 101 | 1       | RAG Fundamentals | pdf       | 20         | processed  |
| 102 | 1       | PostgreSQL Guide | pdf       | 45         | processed  |
| 103 | 2       | Python Basics    | pdf       | 30         | processing |
| 104 | 3       | FastAPI Tutorial | md        | 15         | failed     |

| id  | user_id | title            | file_type | page_count | status     |
|-----|---------|------------------|-----------|------------|------------|
| 105 | 4       | Vector Databases | pdf       | 50         | processed  |
| 106 | 5       | Docker Guide     | pdf       | 25         | processing |

And:

**users**

| id | name  | country | account_status |
|----|-------|---------|----------------|
| 1  | Aarav | India   | active         |
| 2  | Emma  | USA     | active         |
| 3  | Liam  | UK      | inactive       |
| 4  | Sofia | India   | active         |
| 5  | Noah  | USA     | suspended      |

### 3. The Problem GROUP BY Solves

Suppose you run:

```
SELECT COUNT(*)
FROM documents;
```

You get:

6

But what if you want:

processed → ?

processing → ?

failed → ?

A normal COUNT() doesn't tell you that.

You need to divide the rows into groups.

That's what GROUP BY does.

---

### 4. Basic GROUP BY

Start with:

```
SELECT
status,
COUNT(*)
FROM documents
GROUP BY status;
```

Conceptually PostgreSQL creates groups:

processed

processing

failed

Then COUNT(\*) operates independently on each group.

Result conceptually:

| status     | count |
|------------|-------|
| processed  | 3     |
| processing | 2     |
| failed     | 1     |

The important mental model is:

All rows

↓

GROUP BY status

↓

multiple groups

↓

aggregate each group

---

## 5. GROUP BY Doesn't Mean "Sort"

This is a common beginner mistake.

GROUP BY:

Creates groups for aggregation.

ORDER BY:

Sorts the result.

They solve different problems.

For example:

SELECT

status,

COUNT(\*) AS document\_count

FROM documents

GROUP BY status

ORDER BY document\_count DESC;

Here:

GROUP BY → creates groups

COUNT → calculates per-group counts

ORDER BY → sorts those groups

---

## 6. Always Use an Alias

Instead of:

```
SELECT
status,
COUNT(*)
  FROM documents
  GROUP BY status;
```

prefer:

```
SELECT
status,
COUNT(*) AS document_count
  FROM documents
  GROUP BY status;
```

This produces a useful result:

| status     | document_count |
|------------|----------------|
| processed  | 3              |
| processing | 2              |
| failed     | 1              |

Aliases become especially important when your SQL feeds an API or dashboard.

---

## 7. Grouping by File Type

Suppose you want:

Number of documents by file type.

```
SELECT
file_type,
COUNT(*) AS document_count
  FROM documents
  GROUP BY file_type;
```

Result conceptually:

| file_type | document_count |
|-----------|----------------|
| pdf       | 5              |
| md        | 1              |

This is a very common analytics query.

---

## 8. GROUP BY + SUM

Suppose you want:

Total pages by file type.

```
SELECT
```

```
file_type,  
SUM(page_count) AS total_pages  
FROM documents  
GROUP BY file_type;
```

Now each file type gets its own total.

Conceptually:

pdf → sum PDF pages

md → sum Markdown pages

---

## 9. GROUP BY + AVG

You can calculate an average for each group:

```
SELECT  
file_type,  
AVG(page_count) AS average_pages  
FROM documents  
GROUP BY file_type;
```

Now you're asking:

What is the average document size for each file type?

---

## 10. GROUP BY + MIN / MAX

For example:

```
SELECT  
file_type,  
MIN(page_count) AS minimum_pages,  
MAX(page_count) AS maximum_pages  
FROM documents  
GROUP BY file_type;
```

This can produce:

| file_type | minimum_pages | maximum_pages |
|-----------|---------------|---------------|
| md        | 15            | 15            |
| pdf       | 20            | 50            |

## 11. Multiple Aggregates

You can combine everything:

```
SELECT
```

```
file_type,  
COUNT(*) AS document_count,  
SUM(page_count) AS total_pages,  
AVG(page_count) AS average_pages,  
MIN(page_count) AS minimum_pages,  
MAX(page_count) AS maximum_pages  
FROM documents  
GROUP BY file_type;
```

This gives you a mini analytics report.

---

## 12. Grouping by User

Suppose you want:

How many documents has each user uploaded?

Use:

```
SELECT  
user_id,  
COUNT(*) AS document_count  
FROM documents  
GROUP BY user_id;
```

Conceptually:

user 1 → 2 documents

user 2 → 1

user 3 → 1

user 4 → 1

user 5 → 1

This is extremely useful for:

- user analytics
  - storage usage
  - billing
  - rate limits
  - RAG usage metrics
- 

## 13. Grouping by Multiple Columns

Here's where things get more interesting.

Suppose you want:

Number of documents for each user and each status.

Use:

```
SELECT
user_id,
status,
COUNT(*) AS document_count
  FROM documents
  GROUP BY
user_id,
status;
```

Now the grouping key is:

user\_id + status

Think of it as:

(1, processed)

(2, processing)

(3, failed)

...

Each unique combination becomes a group.

---

## 14. Multiple Columns Mean Combination

This:

```
GROUP BY user_id, status
```

does **not** mean:

```
group by user_id
```

then separately group by status

It means:

Group rows by the unique combination of user\_id and status.

Example:

| user_id | status     |
|---------|------------|
| 1       | processed  |
| 1       | processed  |
| 1       | processing |
| 2       | processed  |

Groups become:

(1, processed)

(1, processing)

(2, processed)

---

## 15. WHERE + GROUP BY

You can filter rows before grouping.

Suppose:

Count processed documents by user.

```
SELECT
user_id,
COUNT(*) AS processed_documents
  FROM documents
 WHERE status = 'processed'
 GROUP BY user_id;
```

Mental model:

All documents

↓

WHERE status = processed

↓

remaining documents

↓

GROUP BY user\_id

↓

COUNT

This order is important.

---

## 16. Another Example

Requirement:

Average page count of PDF documents by user.

You need:

filter PDFs

↓

group by user

↓

average page count

The structure is:

```
SELECT
user_id,
```

```
AVG(page_count) AS average_pages
FROM documents
WHERE file_type = 'pdf'
GROUP BY user_id;
```

---

## 17. The Most Important GROUP BY Rule

Look at:

```
SELECT
status,
COUNT(*)
FROM documents
GROUP BY status;
```

Why can we select status?

Because:

status

appears in:

```
GROUP BY status
```

And:

```
COUNT(*)
```

is an aggregate.

So every selected expression is either:

1. grouped, or
  2. aggregated.
- 

## 18. Why This Is Invalid

Remember Day 4:

```
SELECT
title,
COUNT(*)
FROM documents;
```

The problem was:

title isn't aggregated and isn't grouped.

Now we could fix the logical structure with:

```
SELECT
title,
```

COUNT(\*)

FROM documents

GROUP BY title;

Now each title becomes its own group.

But if every document has a unique title, every count would probably be:

1

That's why you need to think carefully about **what business dimension you're grouping by**.

---

## 19. Another Important Example

This:

SELECT

user\_id,

status,

COUNT(\*) AS document\_count

FROM documents

GROUP BY user\_id, status;

is valid.

But this:

SELECT

user\_id,

status,

COUNT(\*)

FROM documents

GROUP BY user\_id;

is problematic.

Why?

You're grouping by:

user\_id

but asking PostgreSQL to give you one status.

A user could have:

processed

processing

failed

Which status should PostgreSQL choose?

There isn't a single obvious answer.

---

## 20. GROUP BY and ORDER BY

These are commonly combined.

Example:

Show document counts by status, largest count first.

SELECT

status,

COUNT(\*) AS document\_count

FROM documents

GROUP BY status

ORDER BY document\_count DESC;

Pipeline:

WHERE

↓

GROUP BY

↓

aggregate

↓

ORDER BY

We'll later add HAVING.

---

## 21. RAG Analytics Example

Imagine your production dashboard needs:

How many documents exist in each processing state?

You can group:

processed

processing

failed

Then count.

This gives the ingestion team a quick view of pipeline health.

Another useful metric:

How many pages has each user uploaded?

Group by:

user\_id

and sum:

page\_count

This could support usage limits or billing.

---

## 22. Debugging Challenge #1

Developer writes:

```
SELECT
status,
COUNT(*) AS document_count
  FROM documents;
```

They expect:

processed → 3

processing → 2

failed → 1

### Tasks:

1. Why doesn't this query produce one count per status?
  2. What's missing?
  3. Rewrite it.
- 

## 23. Debugging Challenge #2

Developer writes:

```
SELECT
user_id,
status,
COUNT(*) AS document_count
  FROM documents
 GROUP BY user_id;
```

### Tasks:

1. Why is this invalid?
  2. What information is missing from GROUP BY?
  3. Rewrite it.
- 

## 24. Debugging Challenge #3

Developer wants:

Count processed PDFs by user.

They write:

```
SELECT
```

```
user_id,  
COUNT(*) AS document_count  
FROM documents  
GROUP BY user_id  
WHERE status = 'processed'  
AND file_type = 'pdf';
```

**Tasks:**

1. What's wrong with the clause order?
  2. Rewrite it.
  3. Explain why filtering happens before grouping.
- 

## 25. Exercise Set A — Basic Grouping

### Q1

Count documents by status.

Return:

status

document\_count

---

### Q2

Count documents by file\_type.

---

### Q3

Calculate total pages by file\_type.

---

### Q4

Calculate average page count by file\_type.

---

### Q5

Find the minimum and maximum page count for each file\_type.

---

## 26. Exercise Set B — User Analytics

### Q6

Count documents uploaded by each user\_id.

Return:

user\_id

document\_count

---

### Q7

Calculate total pages uploaded by each user.

---

### Q8

Calculate the average document size for each user.

---

### Q9

Find the largest document uploaded by each user.

You only need to return:

user\_id

largest\_document

**Hint:** You need an aggregate.

---

## 27. Exercise Set C — Multiple Groups

### Q10

Count documents by:

user\_id

status

---

### Q11

Calculate total pages by:

file\_type

status

---

## Q12

Calculate average page count by:

file\_type

status

---

## 28. Exercise Set D — WHERE + GROUP BY

### Q13

Count only processed documents for each user.

---

### Q14

Calculate total pages of PDF documents for each user.

---

### Q15

Calculate average page count of documents that are:

PDF

AND

processed

grouped by user\_id.

---

## 29. Practical Engineering Problem #1

You're building a RAG ingestion dashboard.

Management wants:

For every processing status, show the number of documents and total number of pages.

Return:

status

document\_count

total\_pages

Requirements:

- One SQL query
- Use GROUP BY
- Use appropriate aggregate functions
- Give meaningful aliases
- Sort by document count from highest to lowest

Write it yourself.

---

### 30. Practical Engineering Problem #2

Your RAG application has per-user usage monitoring.

The product team wants:

For every user, show how many documents they've uploaded, the total pages they've uploaded, and their average document size.

Return:

user\_id

document\_count

total\_pages

average\_pages

Write one SQL query.

---

### 31. Practical Engineering Problem #3

Your ingestion team wants:

For each user, show the number of failed PDF documents.

Requirements:

- Only PDFs
- Only failed documents
- Group by user
- Return users with matching documents

Return:

user\_id

failed\_pdf\_count

---

### 32. Production Debugging

A developer writes:

```
SELECT
```

```
file_type,
```

```
status,
```

```
COUNT(*) AS document_count
```

```
FROM documents
```

```
WHERE file_type = 'pdf'
```

```
GROUP BY file_type
```

```
ORDER BY document_count DESC;
```

The requirement is:

Show the number of PDF documents for each status.

**Tasks:**

1. What's wrong?
  2. Which selected column is causing the problem?
  3. What should the GROUP BY contain?
  4. Rewrite the query.
- 

## 33. Interview Questions

Answer these in your own words.

### Q1

What problem does GROUP BY solve?

---

### Q2

What's the difference between:

```
GROUP BY
```

and:

```
ORDER BY
```

---

### Q3

Why does this work?

```
SELECT
```

```
status,
```

```
COUNT(*)
```

```
FROM documents
```

```
GROUP BY status;
```

---

### Q4

Why is this problematic?

```
SELECT
```

```
user_id,
```

```
status,
```

```
COUNT(*)  
FROM documents  
GROUP BY user_id;
```

---

## Q5

What is the difference between:

```
GROUP BY user_id
```

and:

```
GROUP BY user_id, status
```

---

## 34. Explanation Challenge

Explain this query in plain English:

```
SELECT  
user_id,  
COUNT(*) AS document_count,  
SUM(page_count) AS total_pages,  
AVG(page_count) AS average_pages  
FROM documents  
WHERE status = 'processed'  
GROUP BY user_id  
ORDER BY total_pages DESC;
```

Your explanation must cover:

1. Which rows are considered?
  2. How are rows divided into groups?
  3. What is calculated for each group?
  4. How are the groups sorted?
  5. What does one output row represent?
- 

## 35. Data Engineering Thinking

Imagine your RAG system has:

10 million documents

You execute:

```
SELECT  
user_id,  
COUNT(*) AS document_count
```

```
FROM documents
GROUP BY user_id;
```

The query might return only:

50,000 rows

but PostgreSQL potentially needs to process a huge number of source rows to calculate those groups.

### Question

Why can GROUP BY be expensive even though the final result is relatively small?

Think about:

- Scanning rows
- Grouping
- Memory
- Sorting/hashing
- Database resources

You don't need to know PostgreSQL's exact implementation yet.

---

## 36. Day 5 Short Test

### Q1

What does GROUP BY do?

---

### Q2

Write a query to count documents by status.

---

### Q3

Write a query to calculate total pages by file type.

---

### Q4

What's wrong with:

```
SELECT
user_id,
status,
COUNT(*)
FROM documents
GROUP BY user_id;
```

---

## Q5

What is the difference between:

`GROUP BY user_id`

and:

`GROUP BY user_id, status`

---

## Q6

Write a query to count processed documents by user.

---

## Q7

Write a query to calculate average page count by file type.

---

## Q8

Why does WHERE appear before GROUP BY?

---

## Q9

Write a query that returns:

`status`

`document_count`

`total_pages`

for each status.

---

## Q10

Write a query that returns:

`user_id`

`document_count`

`total_pages`

for processed documents only, ordered by total pages descending.

---